BACKEND ENGINEERING · HLD INTERVIEW PREP

JWT

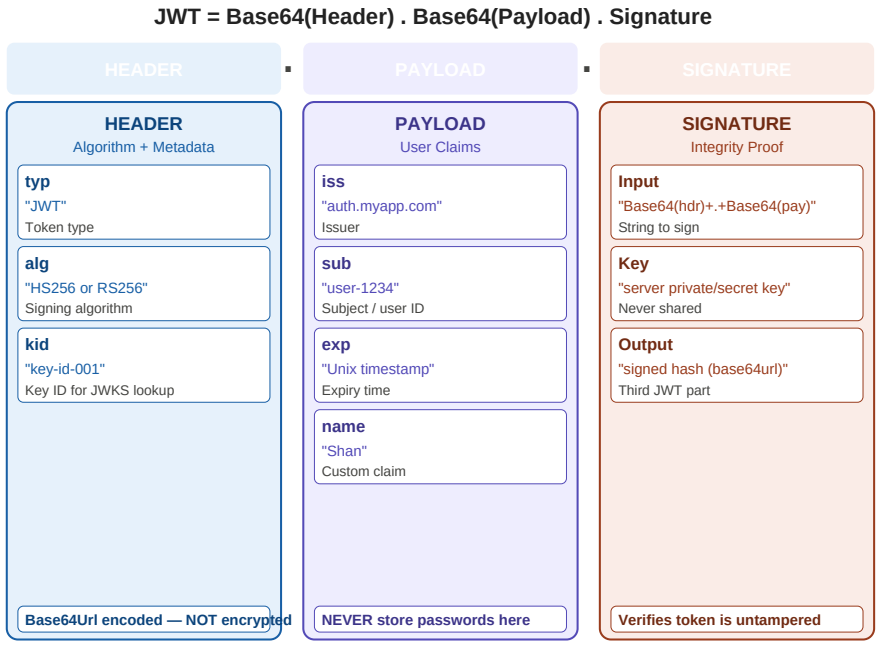
JSON Web Tokens — Study Notes

Structure · Claims · Authentication Flow · Session vs JWT
Token Invalidation · Security Threats · SSO · Best Practices

TOPICS COVERED

1. JWT Structure — Header . Payload . Signature

JWT (JSON Web Token) is a compact, URL-safe token for securely transmitting information as a JSON object. It is **digitally signed** (not encrypted by default) ensuring integrity and authenticity. Format: **Base64Url(header) . Base64Url(payload) . Signature**



Header: algorithm + type · Payload: user claims + metadata · Signature: signed hash proving integrity

Base64Url encoding is NOT encryption. Anyone with the token can decode the header and payload. NEVER put passwords, SSNs, or other secrets in the payload.

The signature IS the security: it proves the token was issued by the server holding the private/secret key and has not been tampered with since.

kid (Key ID): optional field in header. Tells the receiver which public key to use for verification — essential for key rotation and JWKS endpoints.

JWE (JSON Web Encryption): encrypts the payload for confidentiality. Different from JWT which only signs. Use JWE when payload must stay confidential.

2 & 3. Claims + Auth Flow + Session vs JWT

Payload Claims — Three Types

Claim Type	Definition	Examples	Security Note
Registered	Predefined JWT standard fields	iss, sub, aud, exp, nbf, iat, jti	Always include exp — tokens must expire
Public	Custom claims for multiple parties	email, name, country, roles	Register at IANA if broadly shared
Private	App-internal custom claims	internal_user_id, department	Known only to issuer + consumer

Registered Claim	Meaning	Example Value
iss	Issuer — who created the token	"auth.myapp.com"
sub	Subject — who the token is about	"user-1234567"
aud	Audience — intended recipient	"myapp.com"
exp	Expiry — Unix timestamp when token expires	"1716000000"
iat	Issued-at — when token was created	"1715996400"
nbf	Not-before — token invalid before this time	"1715996400"
jti	JWT ID — unique identifier for this token	"uuid-abc-123"

JWT Authentication Flow

7-Step Flow (Instagram + Gmail example)

1	Client	->	Auth Server	POST /login — sends username + password
2	Auth Server	->	(self)	Verify credentials against user database
3	Auth Server	->	Client	Return signed JWT token (access + refresh)
4	Client	->	App Server	GET /resource — Authorization: Bearer
5	App Server	->	Auth Server	Verify token (validate signature / call API)
6	Auth Server	->	App Server	Token valid — confirm to application server
7	App Server	->	Client	Return requested resource — access granted

SSO: same JWT accepted by App1, App2, App3 — each verifies independently. No re-login per app.

Session ID vs JWT

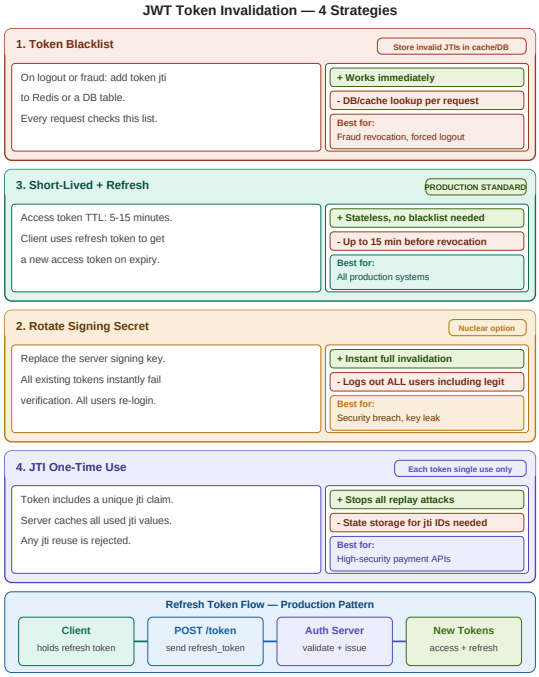
Feature	Session ID	JWT
Storage	DB — session data server-side	None — token is self-contained
State	Stateful — DB hit per request	Stateless — no DB needed
Scalability	Hard — must sync DB	Easy — any server verifies
Revoke	Delete DB record — instant	Blacklist jti or wait for expiry
Overhead	DB query every request	Cryptographic check only

Header: Authorization: Bearer eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1c2VyLTEyMyJ9.signature

Left: 7-step JWT auth flow with Auth Server + App Server · Right: Session ID vs JWT comparison · Bottom: Bearer token header format

4. Token Invalidation — The Hardest JWT Problem

JWT's statelessness is its greatest strength and its biggest weakness. Since the server stores no token state, **invalidating a specific token before its expiry is not straightforward**. Four strategies exist, each with tradeoffs.



Blacklist (immediate but DB overhead) · Rotate secrets (nuclear option) · Short-lived (production standard) · JTI one-time (replay prevention)

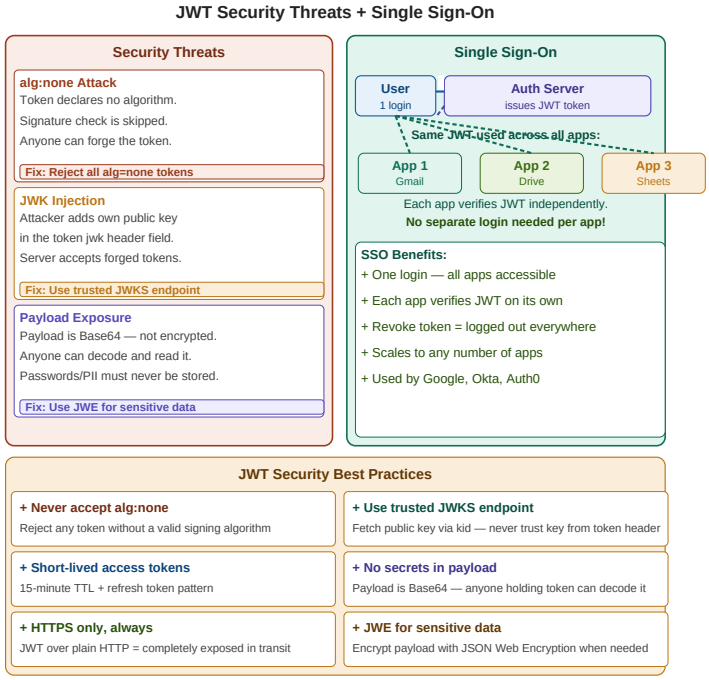
Strategy	Speed	Stateless?	Use Case	Tradeoff
Token Blacklist	Immediate	No — DB/cache hit	Fraud, admin revoke specific user	Reintroduces DB lookup per request
Rotate Secrets	Immediate	Yes	Security breach — invalidate everything	All legitimate users logged out
Short-Lived + Refresh	5-15 min lag	Yes — best practice	Standard production pattern	Token refresh logic complexity
JTI One-Time Use	Immediate	No — store used JTIs	High-security single-use APIs	State storage of used JTI IDs

Production recommendation: Short-lived access tokens (15 min) + long-lived refresh tokens (7-30 days). Refresh tokens stored securely server-side (HttpOnly cookie or secure storage).

On logout: delete refresh token from server. Next access token expiry (max 15 min) = fully logged out. No blacklist needed for most use cases.

High security (banking, healthcare): combine short-lived tokens + JTI blacklist. Every used JTI stored in Redis with TTL matching token expiry.

5 & 6. Security Threats, Best Practices & SSO



Left: 3 security threats with fixes · Right: SSO — one login, multiple apps verify JWT independently · Bottom: 6 best practices

Threat	Attack Vector	Impact	Fix
alg:none	Token header sets alg=none — no signature	Server skips verification — any token accepted	Always reject tokens where alg=none
JWK Injection	Attacker embeds own public key in jwk header	Server uses attacker key — forged token accepted	Only use trusted JWKS endpoint + kid lookup
Payload Exposure	Payload is Base64 — anyone can decode it	Sensitive data (passwords, PII) exposed	Never put secrets in payload. Use JWE to encrypt.
Token Replay	Stolen token reused before expiry	Full session hijack	Short TTL + JTI one-time use + HTTPS only

7. Quick Revision & Interview Questions

Quick Revision

JWT Compact, URL-safe, digitally signed JSON token for secure info transmission	**Structure** Base64Url(header) . Base64Url(payload) . Signature — three dot-separated parts
Header Contains: typ=JWT, alg=HS256/RS256, kid=key ID for JWKS lookup	**Payload** Contains claims. Three types: Registered, Public, Private
Signature HMAC or RSA signature over header+payload — proves integrity, not encryption	**Encoding** Base64Url is NOT encryption — anyone can decode payload. No secrets in payload.
Stateless No server DB needed to verify — just recompute and check signature	**Session vs JWT** Sessions: DB lookup per request. JWT: stateless, no DB hit, scales easily
Bearer token Header: Authorization: Bearer . Different from Basic auth.	**Registered claims** iss=issuer, sub=subject, exp=expiry, iat=issued-at, jti=unique ID
Invalidation JWT valid until exp — cannot revoke without blacklist or rotating key	**Blacklist** Store jti in Redis, check per request. Immediate but defeats statelessness
Short-lived 15 min access token + long-lived refresh token = production standard	**alg:none** Always reject tokens with no algorithm — zero signature = zero security
JWK injection Attacker embeds own public key in header. Fix: only use trusted JWKS + kid	**JWE** JWT with encrypted payload. Use when payload must stay confidential
kid Key ID in header — tells verifier which public key to use from JWKS endpoint	**SSO** User logs in once — same JWT accepted by App1, App2, App3 independently

Interview Questions

Q1. What is JWT? What are its three parts?	**Q2.** Is Base64Url encoding the same as encryption?
Q3. What are the three types of JWT claims?	**Q4.** Walk through JWT authentication flow end to end.
Q5. Session auth vs JWT auth — key differences?	**Q6.** How does JWT enable stateless authentication?
Q7. What is the token invalidation problem in JWT?	**Q8.** What is the alg:none attack and how to prevent it?

Q9. What is JWK injection and how to defend against it?	**Q10.** What is the kid claim and why is it needed?
Q11. What is JWE? When would you use it over JWT?	**Q12.** How does JWT enable SSO across multiple apps?
Q13. Why use short-lived tokens with refresh tokens?	**Q14.** What claims would you put in a JWT for an e-commerce app?
Q15. How would you handle JWT revocation in banking?	

Key Terms

JWT JSON Web Token — compact signed JSON for secure transmission	**JWS** JSON Web Signature — JWT with digital signature
JWE JSON Web Encryption — JWT with encrypted payload	**Header** First segment: alg, typ, kid
Payload Second segment: claims about the user	**Signature** Third segment: signed hash, proves integrity
Registered Claims iss, sub, aud, exp, iat, nbf, jti	**jti** JWT ID — unique token identifier, used for one-time tokens
kid Key ID — identifies public key in JWKS endpoint	**JWKS** JSON Web Key Set — endpoint serving trusted public keys
Bearer Token Authorization: Bearer HTTP header scheme	**Stateless** No server storage needed — token is self-contained
SSO Single Sign-On — one JWT for many apps	**alg:none** Unsigned JWT — ALWAYS reject these
JWK Injection Attack: attacker embeds own public key in token header	**Refresh Token** Long-lived token to get new access tokens after expiry